

Shift Handover Intelligence

An AI agent that interrogates the outgoing shift instead of transcribing it

Field	Detail
Track	Track 1: AI Agent Studio, MCP and multi-agent orchestration
Platform	Freshservice AI Agent Studio (Freddy AI)
Event	The Great Agent Hackathon, The Great Product Festival 2026
Stage 1 deliverables	Written submission, 90-second demo video, Figma prototype

The problem

Shift change takes up less than five percent of operational staff time and accounts for forty percent of plant incidents. In the process industry, roughly every second incident traces back to a communication failure at handover.

Three of the worst industrial inquiries on record say the same thing. At Piper Alpha in 1988, where one hundred and sixty-seven people died, the Cullen inquiry found that a removed pressure safety valve and the blind flange fitted in its place were never communicated between shifts. No written handover procedure existed, and what got recorded was left to each operator's discretion. At Texas City Refinery in 2005, investigators described a total failure of shift handover management: no procedure in use, the lead operator absent, logbooks missing required detail. At Buncefield, supervisors were confused about which tank was being filled.

Today eighty percent of facilities still run unstructured logbooks. Three shifts a day, three hundred and sixty-five days a year, works out at one thousand and ninety-five handovers per line per year, almost all of them verbal or free text, at the moment the plant is statistically most dangerous.

We started from one question. Why hasn't forty years of software fixed this?

The insight

Because the diagnosis has been wrong. Every product in this space treats handover as a documentation problem and ships a better logbook: structured fields, mandatory sections, mobile capture, electronic signatures. None of it works, because handover is a problem of selective attention rather than of record keeping.

The outgoing supervisor writes down what is salient to them, and cannot know what will turn out to matter to the person arriving. A free text logbook makes that worse rather than better, because it is infinitely permissive. It will accept “quiet shift, nothing to report” from someone who left a safety valve out.

You cannot fix selective attention by improving the form. You fix it by putting something in the room that has already read the record and is not relying on recall. An agent inverts the relationship. Because it can see the shift's actual record, every ticket raised, every change

scheduled for the incoming window, every problem still open, it can interrogate the outgoing supervisor rather than transcribe them.

“You logged two stoppages on CBE-L3 this shift, both fault 4021. Same root cause or separate events? I'm asking because Shift C has a tooling changeover on that line at 02:00, and a changeover power-cycles the interlock chain.”

No logbook can ask that question. That behaviour is the entire product.

What it does

Shift Handover Intelligence runs the handover conversation from both sides of the boundary. With the outgoing supervisor it opens by stating what it found, then asks about the specific things it can see. It refuses “nothing to report” on any shift where the record shows a stoppage, an open maintenance job or a scheduled change. It asks one question at a time, classifies each item against the plant's carryover policy, and will not create the handover record without an explicit human confirmation.

With the incoming supervisor it delivers a risk-ordered briefing, states plainly what is not known, and will not close the handover until every Category 1 and Category 2 item is explicitly acknowledged.

In between, it finds what nobody is looking for: a collision. A collision is an unresolved carryover sharing a line or a circuit with scheduled activity in the incoming window. Neither supervisor has any reason to spot one. The outgoing supervisor is thinking about what happened, the incoming one about what is planned, and nobody is thinking about the overlap.

In the demonstration scenario, on line CBE-L3:

Time	Record	Logged by
16:20	Line stops. Fault 4021, gate interlock. Twelve minutes lost.	Operator A
19:45	Line stops. Fault 4021 again. Eighteen minutes lost.	Operator B, separate ticket
20:10	Maintenance opens the gate housing to inspect switch S2-B. Shift ends. Job still open.	Maintenance
02:00	Scheduled: CBE-L3 tooling changeover, which power-cycles the interlock chain.	Planning

Three records, three different people, three different systems of record, each of them individually unremarkable. The agent is the only participant in the handover that sees all three at once.

Carryover risk model

The agent classifies against a policy held in the Freshservice knowledge base, so its judgement comes from the plant's own rules rather than from improvisation.

Category	Meaning	Acknowledgement required
1	Maintenance in progress across the shift boundary	Yes
2	Recurring or unresolved fault	Yes
3	Scheduled activity in the incoming window	No
4	Informational: consumables, calibration, housekeeping	No

A Category 1 or Category 2 item sharing a line with a Category 3 item is a collision, and gets escalated to the top of the briefing regardless of anything else.

How we built it

Entirely on Freshservice AI Agent Studio, with one agent and four bounded workflows.

Workflow	Role
WF1 Start Handover	Retrieve the shift record, open with findings, ask the highest-priority question
WF2 Interrogate Carryovers	Classify against policy, detect and explain collisions
WF3 Commit Handover	Create the handover record, gated on explicit confirmation
WF4 Brief Me	Risk-ordered briefing for the incoming shift, gated on acknowledgement

Layer	Technology
Agent	Freshservice AI Agent Studio (Freddy AI), four workflows across six node types
Records	Freshservice tickets, changes, problems, solution articles
Grounding	Solution articles holding the plant's handover and carryover policy
Integration	Custom API actions over the Freshservice REST API v2
Stage 2	Freshworks FDK 10.x AI Actions app, Node 24, platform v3.0, sixty-two tests at 98.63 percent statement coverage
Demo corpus	Idempotent Python seeder with dry-run mode and a live capability

Layer	Technology
	probe

Why one agent rather than several

Track 1 names multi-agent orchestration, and the honest answer is that Agent Studio has no agent-to-agent handoff. Its handoff node transfers to human agents only.

That constraint turns out to match the evidence. Production surveys of multi-agent systems in 2026 report that the free-form patterns fail hard. Dynamic handoff produces infinite loops and compounding context loss. Multi-agent debate produces sycophancy cascading, where agents reinforce one another into confidently wrong answers. Orchestrator-worker survives because delegation stays constrained.

So this is orchestrator-worker: one agent, four workflows with disjoint intent triggers, routed by the model. Two genuinely distinct roles, a capture agenda in WF1 to WF3 and a synthesis agenda in WF4, implemented as bounded workflows rather than as separate agents passing context between them. We would defend that choice on a platform that did offer free-form handoff.

Why the platform's hardest constraint does not bite

Agent Studio agents are reactive only. They cannot be triggered by system events, which for most agent ideas is a wound you engineer around. Shift handover is naturally two-sided and both sides are human-initiated. The outgoing supervisor starts a handover because their shift is ending. The incoming supervisor asks for a briefing because theirs is beginning. The platform's sharpest limitation simply is not a limitation for this problem, and we chose the problem partly for that reason.

Challenges we ran into

We audited the platform before designing anything, and it saved the project. Probing the live instance's API rather than trusting the documentation turned up CMDB, assets, products, applications and projects all plan-gated and unreachable. Our first instinct had been an asset-centric idea, and it would have been undemonstrable. Changes, problems and releases were open and untouched by any of the twelve built-in actions or any of the twenty pre-built workflows, which is why the design lives there.

The service catalogue API is read-only on this plan. Categories can be created, items cannot, so seeding had to route around it.

Freshservice change management runs a state machine. Change creation rejects any status that is not the configured entry state. Nothing in the API reference says so; it only surfaces against a real instance.

Agent Studio cannot call the Freshservice API without an explicitly configured authentication header, and a silently unset shell variable produces a well-formed header that authenticates as nobody, returning an access denied error rather than an invalid credentials error. That cost us real time and is worth writing down for anyone who follows.

The API action payload editor validates JSON on every variable insert. Insert a property into incomplete JSON and the whole body silently collapses to an empty object. The order that works is: declare the inputs, type complete valid JSON with placeholder words, then replace each placeholder with a property chip. None of that is documented.

Response instructions cap at five hundred characters and total agent instructions at two thousand five hundred. The interrogation agenda had to compress to one thousand eight hundred and thirty-three characters without losing the four-test ordering that makes the agent behave. Most of the design effort in this project went into that text rather than into nodes.

We were also wrong about the platform once, in our own documentation. We had written that Agent Studio cannot do arithmetic or compare dates. It can. There is an Execute Function node and a library of fourteen pre-built functions including arithmetic expressions, date comparison and day counting. We found it mid-build, corrected the claim, and narrowed the Stage 2 argument to what survives scrutiny. A weaker argument that is true beats a strong one a judge can falsify in thirty seconds.

What we are proud of

The agent found all three planted traps unprompted. Given only six words, it returned the open maintenance job first as a Category 1 item, grouped both fault 4021 stoppages as repeat faults, went and found four earlier occurrences of the same fault that month that we had never asked about, and closed with a single sharp question.

It used the category names from the knowledge base policy, which means the grounding is live rather than decorative.

It refuses. Told “quiet shift, nothing to report” on a shift with an open maintenance job, it does not accept it. That is the test we care most about, because it is the exact failure mode Piper Alpha's inquiry describes.

The Stage 2 application is real code rather than a roadmap slide: sixty-two tests, 98.63 percent statement coverage, clearing the packaging coverage gate.

What we learned

Constraints are design information. Every limitation we hit, reactive-only triggers, no agent-to-agent handoff, no iteration, narrowed the space of good ideas rather than the space of possible ones. The idea we shipped is one the platform is unusually well shaped for, and we found it by taking the constraints seriously instead of looking for workarounds.

The prompt is the product. The nodes in these four workflows are almost trivial: collect, call, respond. Every behaviour a judge would notice comes out of one thousand eight hundred and thirty-three characters of instruction. Agent quality turned out to be a writing problem rather than an engineering one.

Verify the platform, not the documentation. Four of the five hardest problems in this build were things the documentation did not say and only a live instance would tell us.

What comes next

Stage 2 is the AI Actions application. The two computations the agent genuinely cannot do, grouping tickets by fault signature and summing downtime per group, and a cross product of carryovers against scheduled work, are already written, tested and packaged as a Freshworks

FDK AI Actions app. It installs as a custom app and appears in the API Action node, so the same agent calls it with no redesign.

That also makes the classification portable. Today the risk model lives in prompt text, which is fine for one plant and wrong for fifty. In the application it becomes versioned code with tests against it.

Beyond that: per-line calibration of the fault signature threshold, handover quality as an operational indicator, and acknowledgement latency as a leading indicator. The gap between the briefing being delivered and the incoming supervisor confirming receipt is exactly the gap Piper Alpha died in.

Why I am the right person to build this

I am a third year chemical engineering student at NIT Tiruchirappalli with a minor in computer science. The chemical engineering half is why I know what a shift handover is. Process safety coursework is where you read the Piper Alpha and Texas City findings, and once you have read them you stop being able to look at a free text logbook and see something harmless.

The rest of my work has been in data, and it has a pattern I only noticed while building this.

At FII Analytics I worked as a data engineering and analytics intern on ISP telemetry. The output that mattered there was a signal quality report that field teams dispatched engineers on, and mean resolution time fell twenty five percent. It was useful because it ended in an instruction rather than a chart.

My machine learning project is an inventory optimisation system built on the M5 dataset. Most forecasting projects stop at an accuracy score, which tells you nothing about whether anyone saved money. So I built a digital twin of a distribution centre, six hundred and fifty nine items with a shelf space cap that binds on every simulated day, and trained a residual PPO agent on top of a base stock policy. It beat base stock by 3.7 percent on the operating objective and won all forty paired out of sample demand draws. The output is a per item order quantity somebody can act on.

Shift Handover Intelligence is the same instinct pointed at a different record. The agent does not summarise the shift. It reads the record and hands the incoming supervisor an instruction he has to acknowledge before the handover closes.

Team

Name	Role	Relevant experience
Jonathan Jerrick	Solo entrant. Problem framing, agent design, workflow build, demo corpus, Stage 2 application.	Third year B.Tech Chemical Engineering with a minor in Computer Science, NIT Tiruchirappalli. Data engineering and analytics at FII Analytics, operations and investment analytics at the Bill and Melinda Gates Foundation, product strategy at Applied AI.

Built with

Freshservice, Freshworks AI Agent Studio, Freddy AI, Freshworks FDK, Node.js, JavaScript, Vitest, Python, REST API v2.

Closing note

Piper Alpha's handover was “complete” too. One shift ended and another began.
Nobody ever confirmed receipt.